

DEEP LEARNING FROM SCRATCH

A Guide for Self-Learners

Episode IV

All Eyes on You

Speed isn't the enemy, confusion is.

Why this mini-guide?

We open its eyes.

Until now, the neuron knew only numbers. Now we give it sight.

For the first time, it will see — and yes, it will stumble.

But here, we awaken that power.

"The most difficult thing is the decision to act; the rest is merely tenacity."

— Amelia Earhart

Pierre Chambet

February 2026

[Launch All Eyes on You here on Google Colab](#)

One click. Your turn to run it.

Here, You Test Our Neuron on Real Images

From Plants to Pixels

In Episode III, we built a neuron that classified plants using two features: leaf length and leaf width. Now we ask: **can the same neuron tackle real images?**

We will use a small dataset of photos — **cats** and **dogs** — and see if the neuron can learn to tell them apart. Same equations. Same learning loop. The only change: each input has **4096 numbers** instead of 2.

You don't need new theory. You already know how the neuron works. What changes is the *shape* of the input — and how we prepare it.

Theory in Short (Reminder)

Nothing changes in the neuron itself. We still have:

1. **Model** — linear transformation $Z = XW + b$, then sigmoid.
2. **Loss** — log-loss measures how wrong the predictions are.
3. **Gradients** — derivatives tell us how to adjust W and b .
4. **Update** — gradient descent: $W \leftarrow W - \alpha dW$, $b \leftarrow b - \alpha db$.

The neuron does not care whether X has 2 columns or 4096. It applies the same math. Our job is to turn a photo into a vector of numbers.

Let's test our neuron on images

We will load cats and dogs, flatten each image into 4096 pixels, normalize them, and feed them to the neuron. Along the way, we will fix a numerical bug, tune hyperparameters, and discover overfitting. **No magic — just the same loop, scaled up.**

*First question every data scientist asks: **what is my dataset?***

Because behind every dataset, there is a goal. Here: can a single neuron learn to distinguish cats from dogs?

The Dataset — Cats and Dogs

We use two separate datasets:

- One for **training** — the neuron learns from it.
- One for **testing** — we evaluate on data the neuron has never seen.

Why two?

If we trained and tested on the same data, the neuron might *memorize* answers without learning to generalize. By testing on **unseen images**, we check if it truly **understands** the task.

It is like preparing for an exam: practicing helps, but the real test is solving **new problems**.

Step 1: Load and Inspect

Step 1: Load the Neuron and the Data

We reuse the code from Episode III and load the cat/dog dataset.

In Colab, you'll import the neuron from Episode III and load the data. One line:

```
X_train, y_train, X_test, y_test = load_data()
```

To inspect what we have — `print(X_train.shape)` — and we see:

Training Data Overview

- `X_train` shape: (1000, 64, 64) — 1000 images, each 64×64 pixels.
- `y_train` shape: (1000, 1) — one label per image.
- Labels: 0 = cat, 1 = dog. Balanced: 500 cats, 500 dogs.

Same idea for the test set:

Test Data Overview

- `X_test` shape: (200, 64, 64) — 200 unseen images.
- `y_test` shape: (200, 1).
- Balanced: 100 cats, 100 dogs.

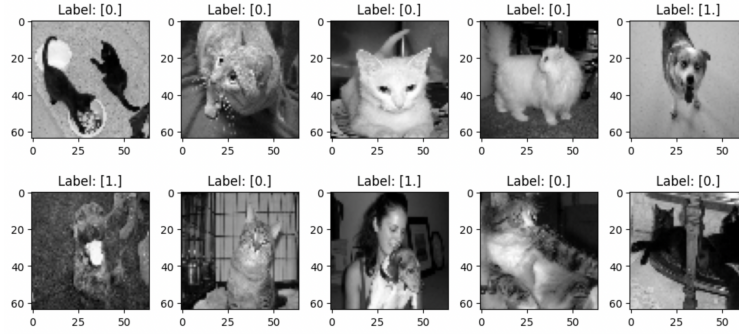


Figure 1: Sample images from the dataset.

Pro tip: Always inspect the shapes before modeling. You want to know exactly what you are feeding to the neuron.

The full setup — imports, paths — is in Colab. Nothing to worry about.

How Do We Feed a Photo to the Neuron?

Now: how do we actually give a photo to the neuron?

The answer is simple. We treat each pixel as a separate input. A 64×64 image has $64 \times 64 = 4096$ pixels.

We scan the image row by row — left to right, top to bottom — and flatten it into a vector:

$$X = (x_1, x_2, \dots, x_{4096})$$

Each x_i is the intensity of one pixel.

A photo becomes a vector of numbers. From there, the neuron applies the same linear model as before.

The neuron doesn't see images — it sees numbers.

The Pipeline: Four Steps

How do we go from raw pixels to a prediction?

1. **Normalize** — rescale each pixel from $[0, 255]$ to $[0, 1]$ (divide by 255 or by max).
2. **Flatten** — turn each 64×64 image into a 4096-dimensional vector.

3. **Train** — same loop as Episode III: model → loss → gradients → update.
4. **Evaluate** — test on unseen data to check generalization.

That is the pipeline. From raw pixels to prediction — four steps.

Step 2: Normalize and Flatten

Step 2: Prepare the Input

Before feeding images to the neuron, we flatten and normalize. Two operations, often done together.

- **Flatten** — each 64×64 image becomes a vector of 4096 values.
- **Normalize** — scale pixels from [0, 255] to [0, 1].

Why divide by 255? Each pixel is stored on 8 bits: $2^8 = 256$ values, from 0 to 255. We divide by the max value — that way it works for any image format.

Why normalize? If one feature has much larger values than others, the model may overweight it. Normalization ensures every feature contributes fairly — like balancing the mix in a band.

We normalize to be fair.

In code — two lines:

```
X_train_reshape = X_train.reshape(X_train.shape[0], -1) / X_train.max()
X_test_reshape = X_test.reshape(X_test.shape[0], -1) / X_test.max()
```

At This Point...

Each image is a normalized vector of 4096 values. The neuron can now learn from the data.

Premium tip: Use -1 in `reshape` to let NumPy infer the dimension. `X.shape[0], -1` means “keep the number of images, flatten the rest.”

Data ready. Time to train.

Step 3: Train — And Hit a Bug

Step 3: Run the Training Loop

We feed the reshaped data to the neuron — same `artificial_neuron` as in Episode III.

```
W, b = artificial_neuron(X_train_reshape, y_train, learning_rate=0.1, num_iter=100)
```

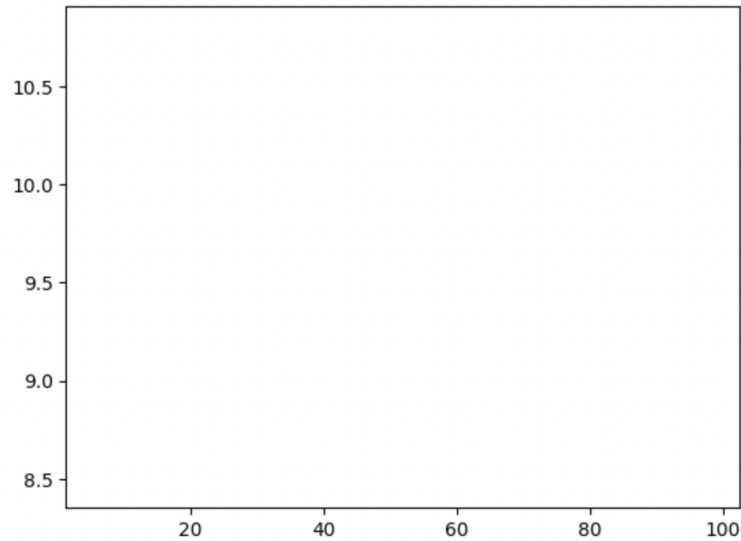


Figure 2: The neuron’s first attempt at seeing — and it stumbles.

We get:

“RuntimeWarning: divide by zero encountered in log”

What is going wrong?

The error comes from `log_loss`. We compute $\log(A)$ and $\log(1 - A)$. A comes from the sigmoid: $A = 1/(1 + e^{-Z})$.

The exponential grows fast. For $z = 20$, we have $\exp(-z) \approx 0$, so $A \approx 1$. For $z = -20$, we have $A \approx 0$. And $\log(0)$ is undefined.

Fix: Add a small constant ϵ to avoid $\log(0)$. Standard practice in deep learning.

```
def log_loss(A, y):  
    m = len(y)  
    epsilon = 1e-15  
    loss = -1/m * np.sum(y * np.log(A + epsilon)  
                        + (1 - y) * np.log(1 - A + epsilon))  
    return loss
```

Pro tip: Numerical stability matters. Always guard against $\log(0)$, division by zero, and overflow. $\varepsilon = 10^{-15}$ is tiny — it does not change the math, but it prevents crashes.

Run the corrected loop in Colab and watch the curve stabilize.

Step 4: Hyperparameters — The Levers We Can Turn

Step 4: Tune the Learning Process

Once the model runs, we control how it learns via **hyperparameters**: learning rate, number of iterations, initialization, etc.

With the fixed `log_loss`, training runs — but the curve may be shaky.

- **Learning rate** — too high $\Rightarrow$ overshoot, instability. Too low $\Rightarrow$ slow convergence.
- **Number of iterations (epochs)** — too few $\Rightarrow$ underfitting. Too many $\Rightarrow$ risk of overfitting.

Hyperparameters = the teacher's schedule. They decide how fast the neuron moves and how long it trains.

In Colab, we lower the learning rate and rerun. The curve smooths out.

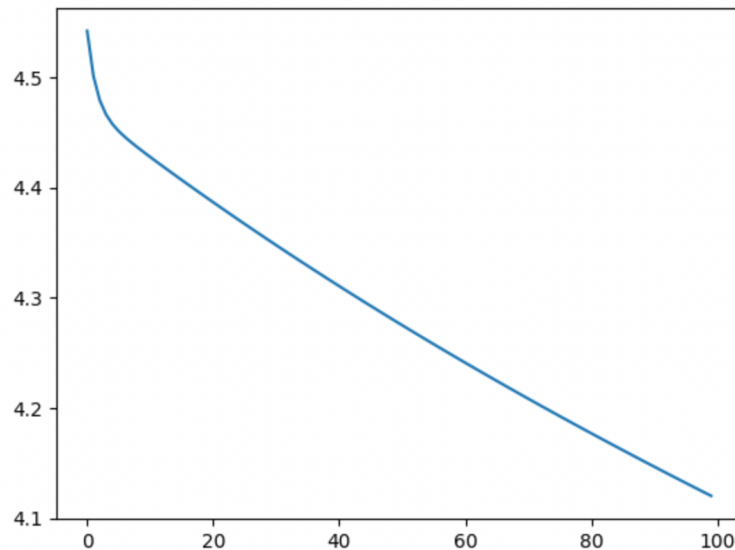


Figure 3: With a gentler learning rate, the curve smooths out.

We can also track both loss and accuracy — and compare training vs. test performance. In Colab, we run a loop that plots them side by side. What we see: the curves diverge.

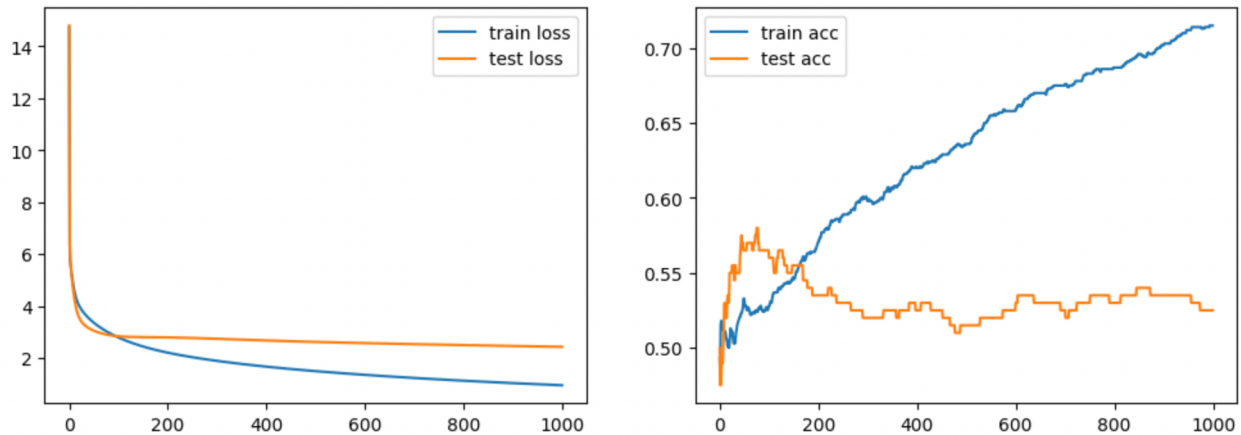


Figure 4: Train climbs. Test lags. The gap tells the story.

Step 5: Detecting Overfitting

Step 5: Diagnose the Divergence

The plots reveal a clear pattern: training loss decreases, training accuracy climbs — but test loss stagnates or rises, test accuracy lags.

Look at the plot above — that gap is the signal. **Overfitting** means the model performs very well on training data but fails to generalize to new examples.

Like a student who memorizes the exercises without understanding the method — they ace the practice set but struggle on new problems.

Warning: if train and test curves diverge, you are likely overfitting.

How Can We Fix Overfitting?

Common strategies:

1. **More data** — expose the model to greater variety.
2. **Simplify the input** — reduce dimensions, extract better features.
3. **Regularization** — L1, L2, dropout — push weights toward simpler solutions.

But here's the twist: in our case, none of these will dramatically help. The problem isn't just overfitting. Why?

Because our model is too simple. We use a **single neuron** — a linear classifier. It can only draw one straight line.

If the true boundary between cats and dogs is curved, complex, nonlinear — a single neuron is not enough.

The Limit — and the Next Step

We have pushed our neuron as far as it can go.

It learned. It adjusted. It gave us decent results on the training set. But it hits a wall: one straight line cannot capture the complexity of images.

What do we do?

We scale up. We add more neurons. We add more layers. We move from a single-cell brain to a full **Artificial Neural Network (ANN)**.

Let's move from a neuron... to a brain.

And yes — that means more equations, matrices, and weight updates. But you only need to understand it once. Then you will be free to design your own architectures.

We gave it eyes. It saw. It stumbled. And now we know why — and what comes next.

In the next episode, we build our first real neural network.